

Interactive Country Explorer

An interactive tool built on the DELVE COVID-19 dataset with below features:

- **Pick any metric** from the dropdown : deaths, cases, totals, the case-fatality ratio, the stringency index, each of the Google mobility trends, or testing, etc.
- **Pick any set of countries** (ctrl-click for several) and overlay them on one chart to compare.
- **Hover** any line to see which country it is.

The controls are pure Bokeh, so it updates instantly.

```
In [3]: import pandas as pd
import numpy as np
from bokeh.io import output_notebook, show
from bokeh.models import ColumnDataSource, Select, MultiSelect, CustomJS, HoverTool
from bokeh.plotting import figure
from bokeh.layouts import row, column
from bokeh.palettes import Category20

output_notebook()

URL = 'https://raw.githubusercontent.com/rs-delve/covid19_datasets/master/dataset/c
df = pd.read_csv(URL, parse_dates=['DATE'])

# (7-day rolling average per country) for the "New deaths (smoothed)" metric below.
df = df.sort_values(['country_name', 'DATE'])
df['deaths_new_smoothed'] = df.groupby('country_name')['deaths_new'].transform(
    lambda s: s.rolling(7, min_periods=1).mean())

# Case-fatality ratio (%): cumulative deaths as a share of cumulative cases.
df['cfr'] = (df['deaths_total'] / df['cases_total'] * 100).replace([np.inf, -np.inf])

# Friendly label
metrics = {
    'New deaths (smoothed)': 'deaths_new_smoothed',
    'New deaths per million': 'deaths_new_per_million',
    'New cases per million': 'cases_new_per_million',
    'Cases (cumulative total)': 'cases_total',
    'Deaths (cumulative total)': 'deaths_total',
    'Case-fatality ratio (%)': 'cfr',
    'Stringency index (0-100)': 'npi_stringency_index',
    'Mobility - retail & recreation': 'mobility_retail_recreation',
    'Mobility - transit': 'mobility_transit_stations',
    'Mobility - workplaces': 'mobility_workplaces',
    'Mobility - residential': 'mobility_residential',
    'Tests (smoothed per 1,000)': 'tests_new_smoothed_per_thousand',
}
cols = list(metrics.values())
```

```

# 40 highest-case countries
top = df.groupby('country_name')['cases_total'].max().nlargest(40).index.tolist()
countries = sorted(top)

# Pack each country's series
data = {}
for c in countries:
    sub = df[(df['country_name'] == c) & (df['DATE'] >= '2020-03-01')].sort_values(
        d = {'date': [int(t.value // 10**6) for t in sub['DATE']]}) # epoch ms for the
    for col in cols:
        s = sub[col] if col in sub.columns else pd.Series([np.nan] * len(sub))
        d[col] = [None if pd.isna(v) else float(v) for v in s]
    data[c] = d

palette = list(Category20[20])
start_metric = 'New deaths (smoothed)'
start_countries = [c for c in ['United Kingdom', 'United States', 'Germany', 'France']]

def build(metric_label, sel):
    col = metrics[metric_label]
    return dict(xs=[data[c]['date'] for c in sel],
                ys=[data[c][col] for c in sel],
                name=list(sel),
                color=[palette[i % len(palette)] for i in range(len(sel))])

source = ColumnDataSource(build(start_metric, start_countries))

p = figure(x_axis_type='datetime', height=430, width=860, title=start_metric)
p.multi_line(xs='xs', ys='ys', line_color='color', line_width=2, source=source, hover_tooltips=[('Country', '@name')], line_policy='nearest')
p.add_tools(HoverTool(tooltips=[('Country', '@name')], line_policy='nearest'))
p.xaxis.axis_label = 'Date'

metric_sel = Select(title='Metric', value=start_metric, options=list(metrics.keys()))
country_sel = MultiSelect(title='Countries (ctrl / cmd-click for several)',
                           value=start_countries, options=countries, size=12)

cb = CustomJS(
    args=dict(data=data, source=source, metric_sel=metric_sel, country_sel=country_sel,
               metrics=metrics, palette=palette, p=p),
    code="""
    const col = metrics[metric_sel.value];
    const sel = country_sel.value;
    const xs = [], ys = [], name = [], color = [];
    for (let i = 0; i < sel.length; i++) {
        const c = sel[i];
        xs.push(data[c]['date']);
        ys.push(data[c][col]);
        name.push(c);
        color.push(palette[i % palette.length]);
    }
    source.data = {xs: xs, ys: ys, name: name, color: color};
    source.change.emit();
    p.title.text = metric_sel.value;
    """)

```

```
metric_sel.js_on_change('value', cb)
country_sel.js_on_change('value', cb)

show(row(column(metric_sel, country_sel), p))
```



Loading BokehJS ...

Did the measures control the outcome?

when a country tightens its rules, does the outcome come down? Pick a country and read it top to bottom.

- **Top panel:** new cases and new deaths per million (both 7-day averages), drawn on a log scale .The dashed stringency line uses the right-hand 0-100 axis.
- **Bottom panel:** the 12 government measures as a heatmap, running from the start January 2020. Darker means stricter.

```
In [4]: import pandas as pd
import numpy as np
from bokeh.io import output_notebook, show
from bokeh.models import ColumnDataSource, Select, CustomJS, HoverTool, FactorRange
from bokeh.plotting import figure
from bokeh.layouts import column

output_notebook()

# Reuses df from the explorer cell (full range from 1 Jan 2020). If you skipped it,
# URL = 'https://raw.githubusercontent.com/rs-delve/covid19_datasets/master/dataset
# df = pd.read_csv(URL, parse_dates=['DATE'])

# (column, label, strictest level) - each row is shaded against its own max.
npi_defs = [
    ('npi_masks', 'Masks', 3),
    ('npi_contact_tracing', 'Contact tracing', 2),
    ('npi_testing_policy', 'Testing policy', 3),
    ('npi_public_information', 'Public info', 2),
    ('npi_international_travel_controls', 'Intl travel', 4),
    ('npi_internal_movement_restrictions', 'Internal movement', 2),
    ('npi_stay_at_home', 'Stay at home', 3),
    ('npi_close_public_transport', 'Close transport', 2),
    ('npi_gatherings_restrictions', 'Gatherings', 4),
    ('npi_cancel_public_events', 'Cancel events', 2),
    ('npi_workplace_closing', 'Workplace closing', 3),
    ('npi_school_closing', 'School closing', 3),
]
labels = [d[1] for d in npi_defs] # bottom-to-top on the y-axis
npi_cols = [d[0] for d in npi_defs]

countries = sorted(df.groupby('country_name')['cases_total'].max().nlargest(40).ind
```

```

def pm7(series):
    """7-day average of a per-million series; drop 0/NaN so the log axis is happy."
    s = series.rolling(7, min_periods=1).mean()
    return [None if (pd.isna(v) or v <= 0) else float(v) for v in s]

data = {}
for c in countries:
    sub = df[df['country_name'] == c].sort_values('DATE')
    date_ms = [int(t.value // 10**6) for t in sub['DATE']]

    cases = pm7(sub['cases_new_per_million']) if 'cases_new_per_million' in sub.col
    deaths = pm7(sub['deaths_new_per_million']) if 'deaths_new_per_million' in sub.
    string = ([None if pd.isna(v) else float(v) for v in sub['npi_stringency_index']
              if 'npi_stringency_index' in sub.columns else [None] * len(sub))

    # Heatmap, resampled to weekly so it stays light and readable.
    wk = sub.set_index('DATE')[npi_cols].resample('W').max()
    hx, hy, ha, hl = [], [], [], []
    for col, lab, mx in npi_defs:
        for ts, lvl in wk[col].items():
            hx.append(int(ts.value // 10**6))
            hy.append(lab)
            if pd.isna(lvl):
                ha.append(0.0)
                hl.append(None)
            else:
                ha.append(float(lvl) / mx)
                hl.append(float(lvl))

    data[c] = dict(date=date_ms, deaths=deaths, cases=cases, string=string,
                  hx=hx, hy=hy, ha=ha, hl=hl)

start = 'United Kingdom' if 'United Kingdom' in data else countries[0]
d0 = data[start]

line_src = ColumnDataSource(dict(date=d0['date'], deaths=d0['deaths'], cases=d0['ca
heat_src = ColumnDataSource(dict(x=d0['hx'], y=d0['hy'], alpha=d0['ha'], level=d0['

# Top: real per-million numbers on a LOG axis (cases sit above deaths, as they shou
top = figure(x_axis_type='datetime', y_axis_type='log', height=300, width=900,
            title='Outcome vs policy - ' + start, y_axis_label='Per million, 7-day
top.line('date', 'cases', source=line_src, color='#e67e22', line_width=2, legend_la
top.line('date', 'deaths', source=line_src, color='#c0392b', line_width=2, legend_l

# Stringency (0-100) on its own right-hand axis.
top.extra_y_ranges = {'str': Range1d(start=0, end=100)}
top.add_layout(LinearAxis(y_range_name='str', axis_label='Stringency (0-100)'), 'ri
top.line('date', 'string', source=line_src, color='#2c3e50', line_width=2, line_das
        y_range_name='str', legend_label='Stringency')

top.legend.location = 'top_left'
top.legend.click_policy = 'hide'
top.add_tools(HoverTool(
    tooltips=[('Date', '@date{%F}'), ('Cases/M', '@cases{0.0}'),

```

```

        ('Deaths/M', '@deaths{0.00}'), ('Stringency', '@string{0.0}']],
        formatters={'@date': 'datetime'}, mode='vline'))

# Bottom: NPI heatmap (darker = stricter).
heat = figure(x_axis_type='datetime', height=360, width=900, x_range=top.x_range,
              y_range=FactorRange(*labels), title='Government measures (darker = st
heat.rect(x='x', y='y', width=7 * 86400 * 1000, height=0.9, source=heat_src,
          fill_color='#08519c', fill_alpha='alpha', line_color=None)
heat.add_tools(HoverTool(
    tooltips=[('Policy', '@y'), ('Date', '@x{%F}'), ('Level', '@level')],
    formatters={'@x': 'datetime'}))
heat.xaxis.axis_label = 'Date'

sel = Select(title='Country', value=start, options=countries)
sel.js_on_change('value', CustomJS(
    args=dict(data=data, line_src=line_src, heat_src=heat_src, top=top),
    code="""
        const d = data[cb_obj.value];
        line_src.data = {date: d['date'], deaths: d['deaths'], cases: d['cases'], s
        heat_src.data = {x: d['hx'], y: d['hy'], alpha: d['ha'], level: d['hl']};
        line_src.change.emit();
        heat_src.change.emit();
        top.title.text = 'Outcome vs policy - ' + cb_obj.value;
    """))

show(column(sel, top, heat))

```



BokehJS 3.9.1 successfully loaded.

A note on reading it: new deaths is the more reliable outcome, because confirmed cases depend heavily on how much a country tested. Cases react faster but are noisier; deaths react slower but are harder to miss.

Mahmoud

In []: